

第 7 讲: 激活函数

从非线性映射到 PINN 中的激活函数选择

@ Frano

● FranSpace

本讲摘要 / Why it matters

激活函数 (Activation Function, AF) 是神经网络中最关键的组成部分之一。它的作用不是“让公式更复杂”，而是为网络引入非线性，使多层神经网络能够逼近复杂函数。如果没有激活函数，那么无论网络叠多少层，本质上仍然只是一个线性变换，表达能力会非常有限。对深度学习初学者来说，理解不同激活函数的公式、图像、优缺点和适用场景，是理解神经网络训练机制的重要一步。对 PINN 而言，激活函数还会进一步影响自动微分的稳定性以及一阶、二阶甚至更高阶导数的质量，因此它不仅影响训练效果，也影响最终物理解的精度。

7.1 为什么需要激活函数

在前一讲中我们已经看到，一个神经元最基本的形式可以写成

$$z = \mathbf{w}^T \mathbf{x} + b.$$

如果直接把 z 作为输出，那么这个神经元做的只是线性映射。若把很多这样的线性层连续叠加起来，整体仍然等价于一个更大的线性变换。也就是说，如果网络中没有激活函数，那么“多层”这一结构并不会真正带来强大的表达能力。

激活函数的核心作用，是把线性变换之后的结果再做一次非线性映射：

$$a = \sigma(z).$$

这一步看起来很简单，却极大改变了网络的能力。引入激活函数之后，网络才有可能表示弯曲的曲线、复杂的边界和高维空间中的非线性关系。

从直观上看，激活函数相当于给神经元增加了“非线性反应机制”。输入很小时，它可能压缩输出；输入变大时，它可能放大响应；有些激活函数会对负值直接截断，有些会保留一部分负信息。这些不同的处理方式，最终会影响网络学习特征的方式。

分析 / Insight

可以把激活函数理解为“神经元的响应规则”。同样的输入经过不同激活函数后，输出的数值范围、变化速度和导数性质都可能不同，而这些差异会直接影响梯度传播、训练稳定性和最终模型性能。

7.2 如何理解激活函数的好坏

一个激活函数是否“好用”，通常可以从以下几个角度理解：

- 是否能引入足够的非线性；
- 是否容易产生梯度消失或梯度爆炸；
- 是否计算简单，训练成本是否可控；
- 是否光滑，是否适合高阶自动微分；
- 输出是否零中心化；
- 是否容易造成“死神经元”问题。

在普通深度学习任务中，大家常常关注训练速度和分类/回归效果；但在 PINN 中，还需要额外关注导数的连续性与平滑性。因此，某些在普通任务中很流行的激活函数，不一定就是 PINN 中最优的选择。

7.3 常用激活函数总览

目前常见的激活函数很多。为了方便整理，本讲将它们分为以下几类：

- 最基础、最早期的激活函数：Binary Step、Linear、Sigmoid、Tanh；
- ReLU 家族：ReLU、Leaky ReLU、PReLU；
- 指数型平滑函数：ELU、SELU、Softplus；
- 其他平滑激活函数：Softsign、SiLU/Swish、GELU、Mish；
- 输出层常用函数：Softmax。

下面逐个进行说明。

7.4 Binary Step

Binary Step 是最早期、最直观的一类激活函数，可以写成

$$\sigma(x) = \begin{cases} 1, & x \geq 0, \\ 0, & x < 0. \end{cases}$$

它的思想非常简单：输入如果超过某个阈值，就输出 1；否则输出 0。这种形式和“是否激活”的直觉很接近，因此在感知机的早期讨论中很常见。

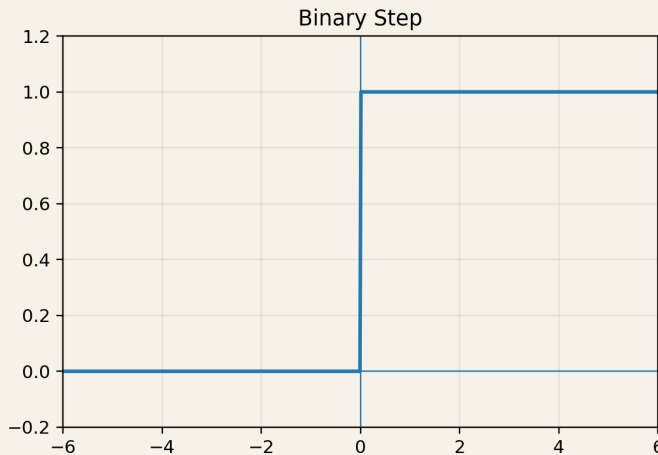


图 1: Binary Step 激活函数示意图

它的优点是概念简单、容易理解，但缺点也非常明显：函数几乎处处不可导，导数要么为 0，要么在跳变点不存在。因此它无法很好地用于基于梯度的反向传播训练。现代神经网络几乎不会在隐藏层中使用它。

7.5 Linear

Linear 就是不加任何非线性处理，直接输出输入本身：

$$\sigma(x) = x.$$

它的优点是形式简单、导数恒为 1，不会产生梯度消失问题。但它最大的问题在于不引入非线性。若网络层层都使用 Linear，那么整个网络退化为一个线性模型。

不过，Linear 在输出层仍然很常见，特别是在回归问题中。当我们希望输出值不受限制地取任意实数时，输出层经常直接使用线性形式。

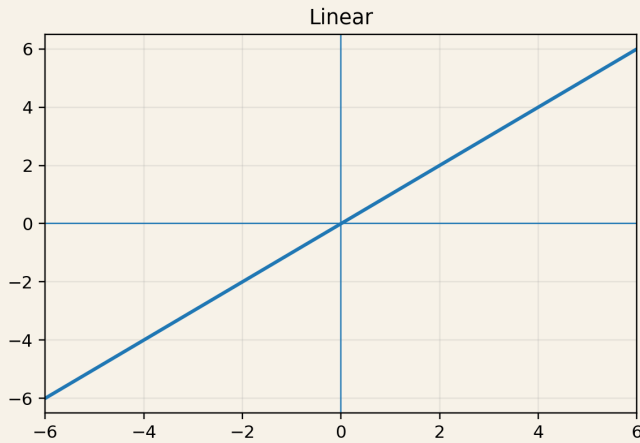


图 2: Linear 激活函数示意图

7.6 Sigmoid

Sigmoid 可以写成

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

它的输出范围是

$$0 < \sigma(x) < 1.$$

Sigmoid 的图像是一条平滑的 S 型曲线。它的优点是输出有明确范围，常常可以解释为概率，因此很适合二分类输出层。例如，在二分类问题中，输出层常用 Sigmoid 把任意实数映射到 0 到 1 之间。

但它也有明显缺点。第一，输入很大或很小时，函数会进入饱和区，这时导数会非常小，容易产生梯度消失。第二，它的输出不是零中心的，即输出总为正，这会影响优化效率。正因为如此，Sigmoid 现在通常不再用于深层隐藏层，但在二分类输出层中仍然很常见。

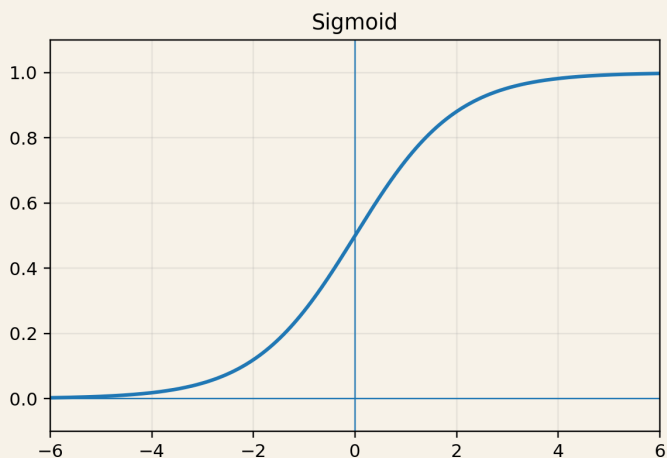


图 3: Sigmoid 激活函数示意图

7.7 Tanh

Tanh 是双曲正切函数:

$$\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

它的输出范围是

$$-1 < \sigma(x) < 1.$$

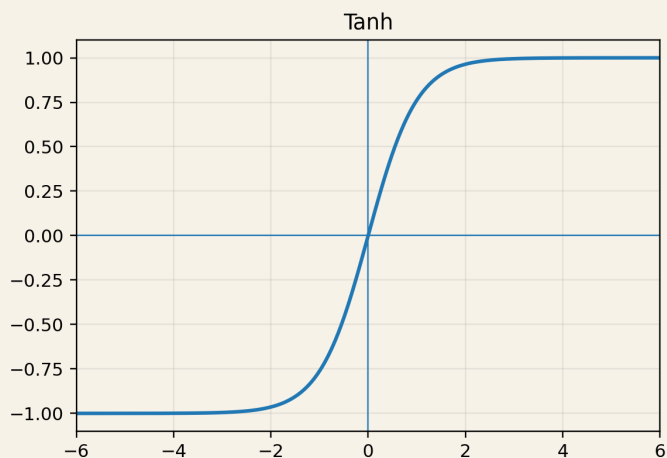


图 4: Tanh 激活函数示意图

Tanh 可以看作是“零中心化的 Sigmoid”。相较于 Sigmoid，它的输出既有正值也有负值，因此在训练时通常更有利于梯度更新。它的图像也同样平滑，因此在需要连续导数的

场景中非常常用。

Tanh 的主要缺点是：当输入绝对值很大时，仍然会进入饱和区，因此也会有梯度消失问题。不过在很多 PINN 工作中，Tanh 依然是非常经典且实用的选择，因为它光滑、导数连续、数值表现稳定，适合计算高阶导数。

备注 / Note

如果只是从“普通深度学习的训练速度”角度看，Tanh 可能不一定是最优选择；但如果从 PINN 所需的高阶自动微分和物理场平滑性角度看，Tanh 往往是非常稳妥的选择。

7.8 ReLU

ReLU 的英文是 Rectified Linear Unit，简称 ReLU，其定义为

$$\sigma(x) = \max(0, x).$$

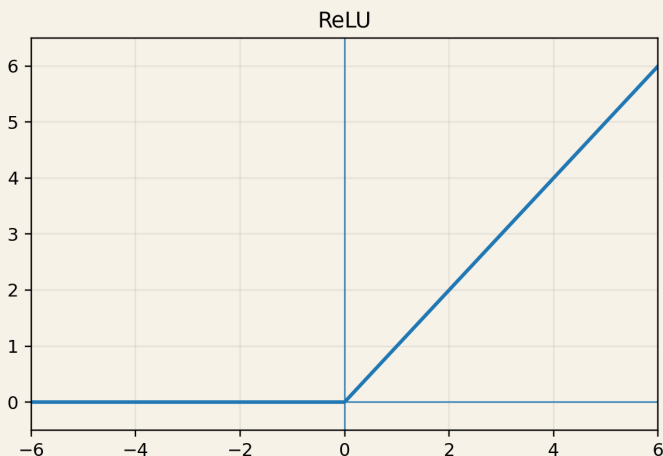


图 5: ReLU 激活函数示意图

ReLU 最大的优点是简单、高效，并且在正半轴上导数恒为 1，不容易像 Sigmoid 和 Tanh 那样出现严重的梯度消失。因此，它成为普通深度学习中最经典、最常用的隐藏层激活函数之一。

但 ReLU 也有两个重要问题。第一，在负半轴上输出恒为 0，导数也为 0，因此某些神经元可能长期不再更新，这就是所谓的“死神经元”问题。第二，ReLU 在 0 点不可导，虽然这在很多普通任务中并不构成大问题，但在 PINN 中，高阶自动微分可能会受到影响，因此通常不是最理想的选择。

7.9 Leaky ReLU

Leaky ReLU 是对 ReLU 的改进。它把负半轴从“完全为 0”改为保留一个很小的斜率：

$$\sigma(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \leq 0, \end{cases}$$

其中 α 通常取一个较小的常数，例如 0.01。

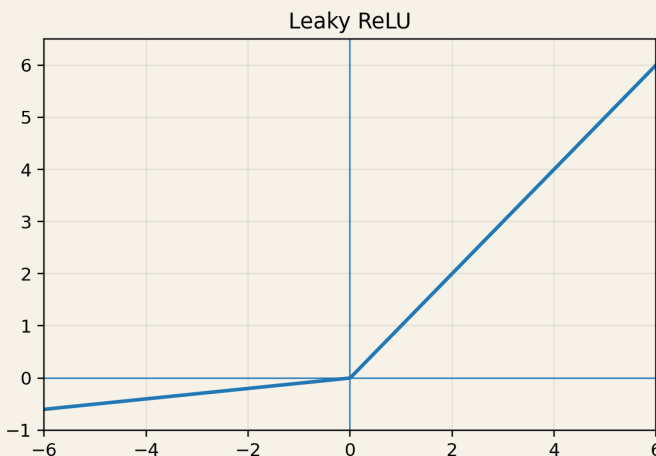


图 6: Leaky ReLU 激活函数示意图

这样做的好处是：即使输入落在负半轴，神经元仍然有一小部分梯度，不至于彻底“死亡”。因此，Leaky ReLU 能在一定程度上缓解 ReLU 的死神经元问题。

它仍然保留了 ReLU 简单高效的优点，但在光滑性方面并没有根本改观，在 0 点仍然不是严格光滑的。因此，在 PINN 中仍然不是最常见的首选。

7.10 PReLU

PReLU 的英文是 Parametric ReLU，它和 Leaky ReLU 的形式相似，但负半轴的斜率不再是预先固定的常数，而是一个可以通过训练学得参数：

$$\sigma(x) = \begin{cases} x, & x > 0, \\ ax, & x \leq 0. \end{cases}$$

PReLU 的优点在于，它比 Leaky ReLU 更灵活。因为斜率 a 不是手动指定的，而是网络自己学出来的，所以模型可以根据数据自动调整负半轴的响应方式。

当然，这也意味着它引入了额外参数，训练时会略微增加复杂度。对于普通深度学习任务，这种灵活性有时是有益的；但在 PINN 中，由于更关心平滑性和导数质量，PReLU 依然不是最常见的主流选择。

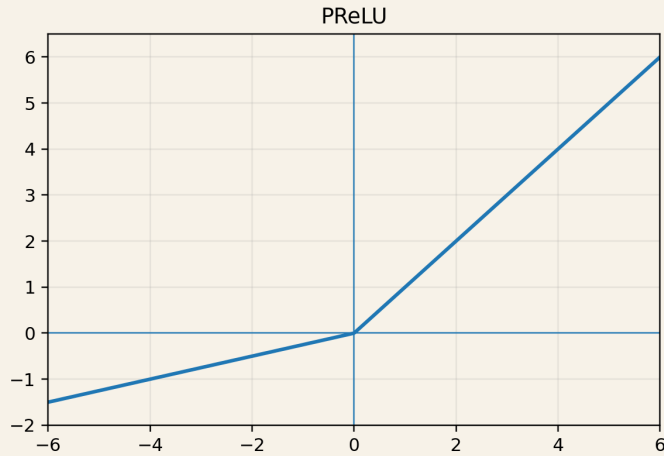


图 7: PReLU 激活函数示意图

7.11 ELU

ELU 的英文是 Exponential Linear Unit, 其定义为

$$\sigma(x) = \begin{cases} x, & x > 0, \\ \alpha(e^x - 1), & x \leq 0. \end{cases}$$

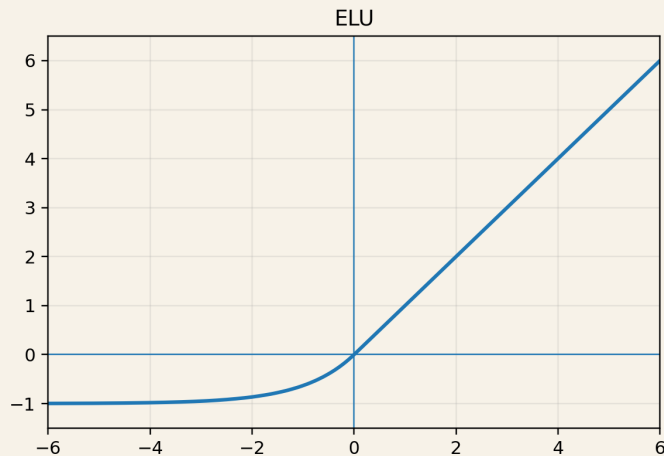


图 8: ELU 激活函数示意图

ELU 在正半轴上与 ReLU 一样保持线性, 在负半轴上则不再是直线, 而是平滑地趋向于一个负值。这带来两个好处: 一是它在负半轴不至于像 ReLU 那样完全失活; 二是它的输出均值更接近零, 有利于训练。

ELU 的计算比 ReLU 更复杂，但它的平滑性更好，因此常被看作是 ReLU 的一种改良版本。

7.12 SELU

SELU 的英文是 Scaled Exponential Linear Unit，它是在 ELU 的基础上再乘上一个缩放系数：

$$\sigma(x) = \lambda \begin{cases} x, & x > 0, \\ \alpha(e^x - 1), & x \leq 0. \end{cases}$$

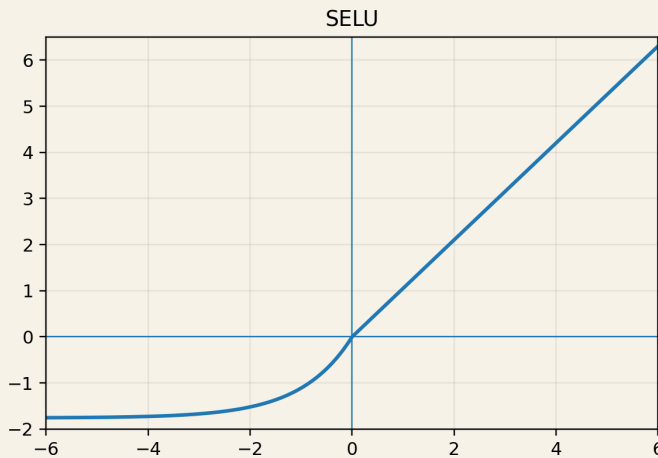


图 9: SELU 激活函数示意图

SELU 的设计目标是让网络在某些条件下具备“自归一化”特性，即层与层之间的输出均值和方差能自动维持在较稳定的范围。这对于深层网络训练是有帮助的。

不过，SELU 的效果通常依赖特定的初始化方式和网络设置，因此使用时有一定条件。相较于 Tanh 和 SiLU，它在 PINN 中并不算主流。

7.13 Softplus

Softplus 是 ReLU 的平滑版本，定义为

$$\sigma(x) = \ln(1 + e^x).$$

当 x 很大时，Softplus 近似于 x ；当 x 很小时，它平滑地接近于 0。和 ReLU 相比，它最大的优点是处处光滑，不会在 0 点出现不可导问题。

从 PINN 的角度看，Softplus 是一个值得关注的候选函数，因为它兼顾了平滑性和一定的 ReLU 风格。但在实际使用中，它不像 Tanh 那么经典，也不像 SiLU 那么流行。

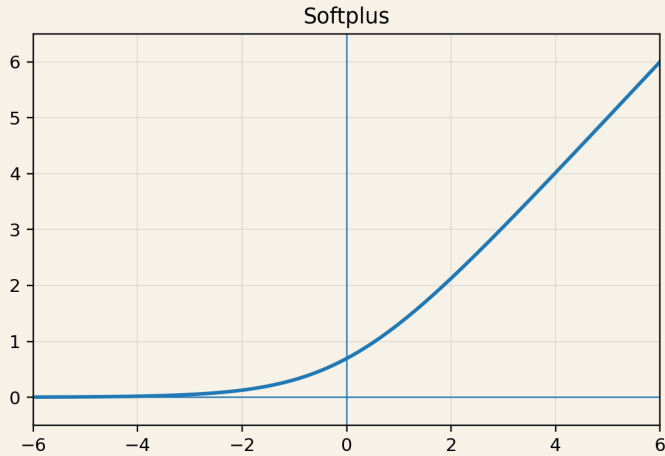


图 10: Softplus 激活函数示意图

7.14 Softsign

Softsign 定义为

$$\sigma(x) = \frac{x}{1 + |x|}.$$

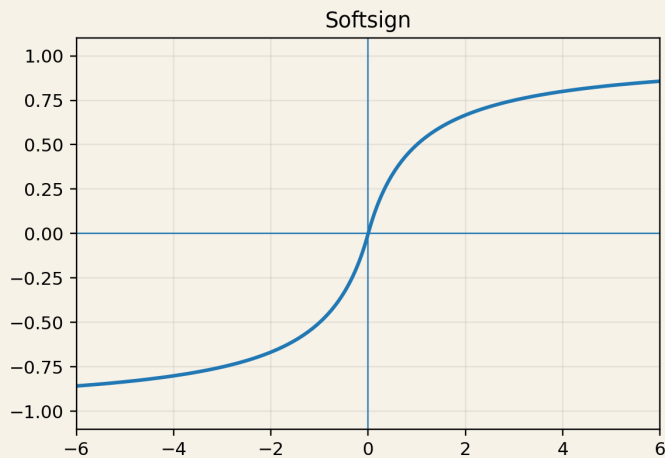


图 11: Softsign 激活函数示意图

它和 Tanh 有点相似，输出范围也在 $(-1, 1)$ 之间，但它的尾部衰减方式不同，变化更缓。Softsign 的优点是公式简单、图像平滑、输出零中心化。它不像 Sigmoid 那样输出全为正值，因此在优化时有一定优势。

不过，相比于 Tanh，Softsign 在实践中不算主流，使用频率较低。

7.15 SiLU / Swish

SiLU 的英文是 Sigmoid Linear Unit，也常被称为 Swish，其定义为

$$\sigma(x) = x \cdot \text{sigmoid}(x).$$

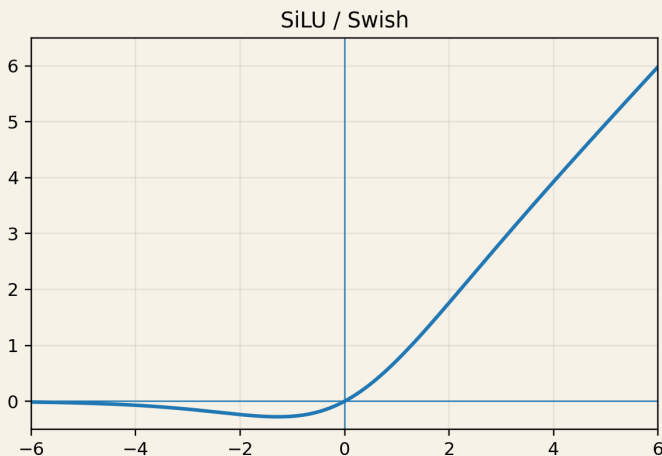


图 12: SiLU / Swish 激活函数示意图

SiLU 的图像是平滑的，在负半轴保留了较小的负输出，在正半轴逐渐接近线性。它兼具平滑性和较好的表达能力，因此近年来在很多深度学习模型中表现很好。

从 PINN 的角度看，SiLU 是一个很有潜力的选择。它比 ReLU 更光滑，又比 Tanh 在某些问题上更灵活，因此在需要稳定高阶导数的任务中也值得尝试。

7.16 GELU

GELU 的英文是 Gaussian Error Linear Unit，其思想可以理解为“按概率平滑地决定一个输入该保留多少”。常见近似写法为

$$\text{GELU}(x) = \frac{1}{2}x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right).$$

GELU 比 ReLU 更平滑，也不像 ReLU 那样简单粗暴地截断负值。它在现代深度学习模型中非常流行，尤其在 Transformer 类模型中很常见。

它的优点是光滑、表达能力好、训练效果常常不错；缺点是计算稍复杂。对于 PINN 而言，它的平滑性是加分项，但相比 Tanh，使用经验和传统积累相对少一些。

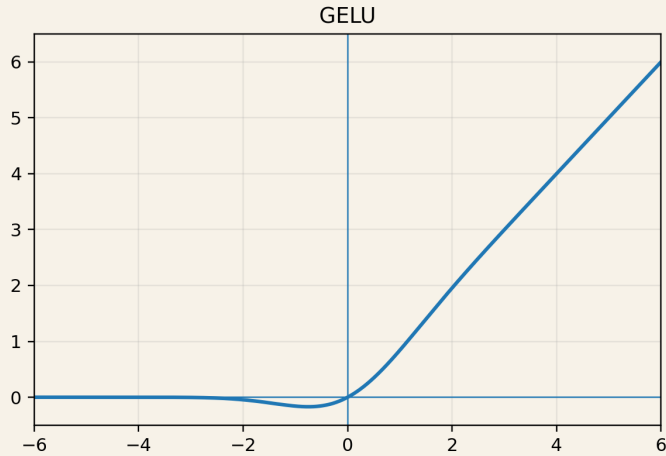


图 13: GELU 激活函数示意图

7.17 Mish

Mish 定义为

$$\text{Mish}(x) = x \tanh(\ln(1 + e^x)).$$

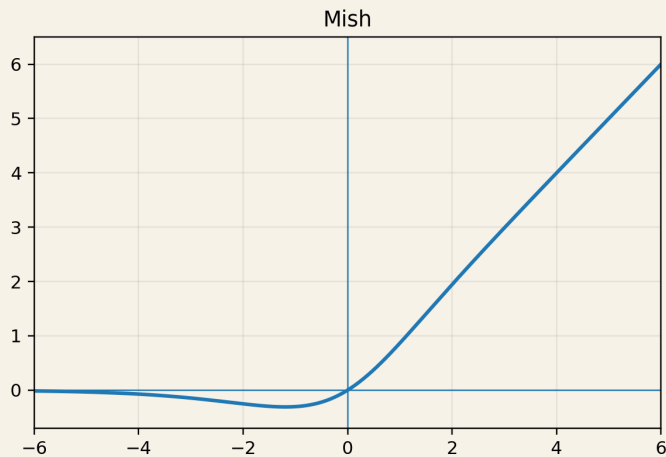


图 14: Mish 激活函数示意图

Mish 也是一种平滑激活函数，图像与 SiLU 有些相似，但非线性形状略有不同。它具有平滑、非单调、对负值较温和等特点，理论上能够在某些任务中取得不错表现。

不过，相比 Tanh、ReLU 和 SiLU，Mish 的使用频率没有那么多，更多见于一些探索性或改进性工作中。

7.18 Softmax

Softmax 通常写成

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

它和前面那些点对点作用的激活函数不同。Softmax 不是对单个标量独立作用，而是对一个向量整体进行归一化，使所有输出分量都为正，并且总和为 1。因此，它特别适合多分类输出层。

为了帮助直观理解，这里给出一个两分类情形下的示意图。若输入向量写成 $(x, 0)$ ，那么 Softmax 的第一个分量会随 x 增大而上升，第二个分量则相应下降。

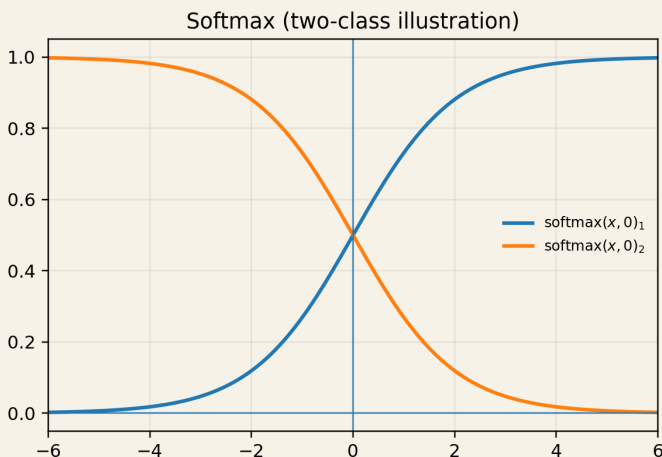


图 15: Softmax 在两分类情形下的示意图

在 PINN 中，Softmax 一般不是隐藏层常用激活函数，但若某些问题需要分类型输出，它仍然是输出层的重要选择。

7.19 PINN 中该如何选择激活函数

对于 PINN 来说，选择激活函数时往往要比普通任务更谨慎。因为 PINN 不只是做前向预测，还要做自动微分，构造 PDE 残差，并且经常需要一阶、二阶甚至更高阶导数。

从这个角度看，常见结论大致如下：

- Tanh：非常经典、稳定，是 PINN 中最常见的选择之一；
- SiLU / Swish：平滑、表现好，也很值得尝试；
- Softplus：平滑，可作为候选；
- GELU、Mish：也具备平滑性，但使用经验相对少；

- ReLU、Leaky ReLU、PReLU：在普通深度学习中很常见，但对 PINN 的高阶导数不一定友好；
- Sigmoid：可以使用，但容易梯度消失，通常不如 Tanh 常用。

备注 / Note

对初学者来说，若你正在写第一个 PINN 程序，并且没有特别强的先验偏好，那么先用 Tanh 往往是最稳妥的起点。等基础程序跑通后，再尝试 SiLU、Softplus 或 GELU 做比较，会更有意义。

7.20 常见误区

关于激活函数，初学者很容易有几个误区：

- 误区一：激活函数只是“可有可无的小细节”。
实际上，它会直接影响网络的表达能力和训练过程。
- 误区二：普通深度学习里常用的 ReLU，在 PINN 中也一定最优。
事实并非如此，因为 PINN 更看重导数的平滑性与高阶自动微分质量。
- 误区三：激活函数越新越复杂越好。
并不是。合适比“新”更重要，很多时候最稳妥的仍然是 Tanh。
- 误区四：所有激活函数都适合放在任何一层。
实际上，隐藏层和输出层的需求不同。比如回归输出层常用 Linear，二分类输出层常用 Sigmoid，多分类输出层常用 Softmax。

7.21 一页总结

- 激活函数的作用是引入非线性，使神经网络能够拟合复杂函数。
- 没有激活函数时，多层网络本质上仍然只是一个线性模型。
- 常见激活函数包括 Sigmoid、Tanh、ReLU、Leaky ReLU、PReLU、ELU、SELU、Softplus、Softsign、SiLU/Swish、GELU、Mish 和 Softmax。
- 对普通深度学习来说，ReLU 及其变体非常常见；对 PINN 来说，Tanh、SiLU、Softplus 这类平滑函数通常更值得优先考虑。
- 回归输出层常用 Linear，二分类输出层常用 Sigmoid，多分类输出层常用 Softmax。

表 1: 常用激活函数总结

函数	数学形式	输出范围	主要优点
Binary Step	$\{0, 1\}$ 跳变函数	$\{0, 1\}$	概念简单
Linear	x	$(-\infty, \infty)$	简单, 导数恒为 1
Sigmoid	$\frac{1}{1 + e^{-x}}$	$(0, 1)$	可解释为概率
Tanh	$\tanh(x)$	$(-1, 1)$	零中心化, 光滑
ReLU	$\max(0, x)$	$[0, \infty)$	简单高效
Leaky ReLU	x 或 αx	$(-\infty, \infty)$	缓解死神经元
PReLU	x 或 ax	$(-\infty, \infty)$	负半轴斜率可学习
ELU	x 或 $\alpha(e^x - 1)$	$(-\alpha, \infty)$	负半轴平滑
SELU	缩放后的 ELU	$(-\lambda\alpha, \infty)$	有自归一化特性
Softplus	$\ln(1 + e^x)$	$(0, \infty)$	ReLU 的平滑版本
Softsign	$\frac{x}{1 + x }$	$(-1, 1)$	平滑、零中心化
SiLU / Swish	$x \cdot \text{sigmoid}(x)$	$(-\infty, \infty)$	平滑、表现好
GELU	平滑概率门控形式	$(-\infty, \infty)$	平滑、表达能力好
Mish	$x \tanh(\ln(1 + e^x))$	$(-\infty, \infty)$	平滑、非单调
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$ 且和为 1	输出概率分布